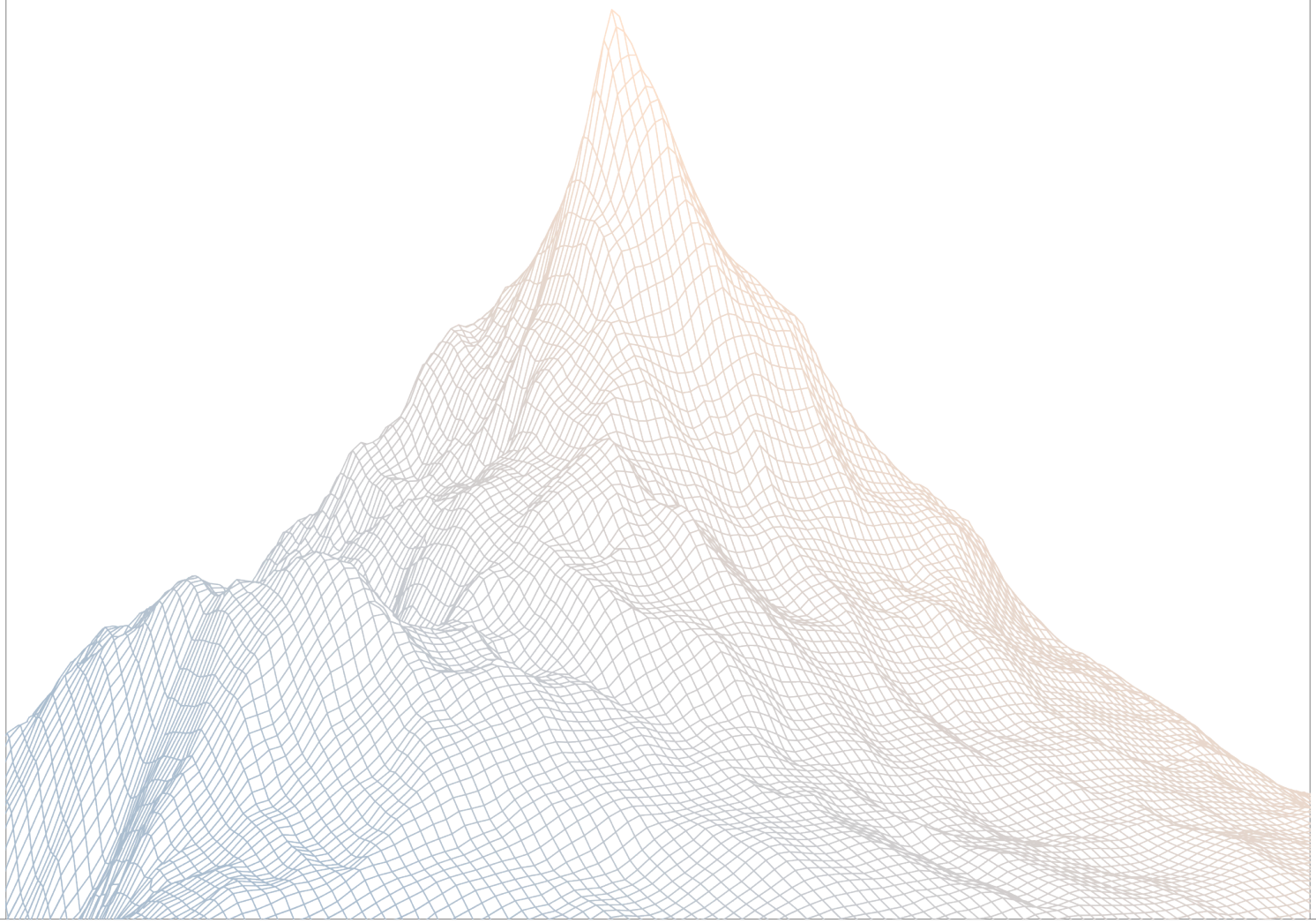
Zenith

Berachain

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: September 9th to September 15th, 2025

AUDITED BY: cccz
eeyore

Contents

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Berachain

Berachain is a high-performance EVM-Identical Layer 1 blockchain utilizing Proof-of-Liquidity (PoL) and built on top of the modular EVM-focused consensus client framework BeaconKit.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts-staking-pools
Repository	https://github.com/berachain/contracts-staking-pools
Commit Hash	`e61cae7e37401920c2ce1047ad308ede018ede57`
Files	`src/**.sol`

2.3 Audit Timeline

September 9, 2025	Audit start
September 15, 2025	Audit end
September 25, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	2
Medium Risk	4
Low Risk	5
Informational	12
Total Issues	**23**

3

Findings Summary

ID	Description	Status
H-1	Triggering fullExit will not fully exit the validator	Resolved
H-2	AccountingOracle.updateTotalDeposits() has no access control	Resolved
M-1	Missing function in SmartOperator prevents IncentiveCollector fee percentage changes	Resolved
M-2	Double fee charging through share dilution after full exit causes user fund loss	Resolved
M-3	BGT balance undervaluation during full exit leads to user fund loss	Resolved
M-4	Share inflation attack allows malicious staking pool creator to steal user deposits	Resolved
L-1	Unfair reward distribution in IncentiveCollector due to incorrect fee accounting order	Resolved
L-2	Dust assets cannot be withdrawn	Acknowledged
L-3	Withdrawals may fail due to exceeding totalDeposits	Resolved
L-4	StBERA.previewRedeem() should round down	Resolved
L-5	Unnecessary _safeMint() can block withdrawals for contracts not supporting onERC721Received()	Resolved
I-1	EOA users unprotected against race conditions in IncentiveCollector claim function	Acknowledged
I-2	Add validation to prevent zero beacon block root usage in proof verification	Resolved
I-3	Incorrect comment and redundant code in accrueEarnedBGTFees() function	Resolved

ID	Description	Status
I-4	Update NatSpec documentation and inheritance docs for consistency	Resolved
I-5	Remove redundant declarations and duplicate imports	Resolved
I-6	Restrict recoverERC20() to only work after full exit in IncentiveCollector contract	Acknowledged
I-7	Missing explicit requestId validation in _finalizeWithdrawalRequest()	Resolved
I-8	Change public functions to external	Resolved
I-9	Use Ownable2StepUpgradeable instead of OwnableUpgradeable	Resolved
I-10	Add clarification that direct BEACON_DEPOSIT donations are considered a loss	Acknowledged
I-11	Incorrect deposit prioritization leads to rewards dilution for existing users	Acknowledged
I-12	The Deployer contract may always fail to deploy	Resolved

4

Findings

4.1 High Risk

A total of 2 high risk findings were identified.

[H-1] Triggering fullExit will not fully exit the validator

SEVERITY: High	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Medium

Target

- AccountingOracle.sol#L28-L46

Description:

When the withdrawal triggers full exit, the following call path will be executed.

```
> WithdrawalVault._withdraw()
> StakingPool.notifyWithdrawalRequest()
> StakingPool._triggerFullExit()
> WithdrawalVault.notifyFullExitFromCL()
```

`notifyFullExitFromCL()` will set `_isFullyExited[pubkey] = true`.

```
function notifyFullExitFromCL(bytes memory pubkey) external {
    // Get the staking pool for this validator to verify the sender
    ICoreContractsStorage.CoreContracts memory coreContracts
        = _factory.getCoreContracts(pubkey);
    if (msg.sender ≠ coreContracts.stakingPool) {
        revert InvalidSender();
    }

    _isFullyExited[pubkey] = true;
}
```

But in `_withdraw()`, it requires `_isFullyExited[pubKey]` to be false before setting `assetsInGWei = 0` and requesting full exit validator in

`_triggerExecutionLayerWithdrawal()`.

```
(uint256 shares, bool isFullExitWithdraw, bool isShortCircuit) =
    IStakingPool(coreContracts.stakingPool).
        notifyWithdrawalRequest(msg.sender, amountInWei);

// Only trigger withdrawals if pool hasn't fully exited and short circuit is
    not triggered
if (!_isFullyExited[pubkey] && !isShortCircuit) {
    // For full exit withdrawals, set amount to 0 and mark as exited
    if (isFullExitWithdraw) {
        assetsInGWei = 0;
    }

    uint256 feeToPay = _getWithdrawalRequestFee();
    if (feeToPay > maxFeeToPay) {
        revert OverpaidFee();
    }

    if (msg.value < feeToPay) {
        revert InsufficientFee();
    }

    _triggerExecutionLayerWithdrawal(pubkey, assetsInGWei, feeToPay);
    _refundOverpaidFee(feeToPay, msg.sender);
```

This prevents the validator from fully exiting, and the Bera held in the validator cannot be withdrawn to the WithdrawalVault.

Recommendations:

It is recommended that `notifyFullExitFromCL()` be called only when `_triggerFullExit()` is being called from `updateTotalDeposits()`. And set `_isFullyExited[pubKey]` in `_withdraw()`.

```
    function _triggerFullExit() internal {
        ISmartOperator(_smartOperator).fullExitQueueDropBoost();
        // Freeze total assets at current value after the drop boost is
    queued and base rate fees are updated.
        _frozenTotalAssets = _getTotalAssets();

        // Transfer buffered assets to the withdrawal vault
        if (bufferedAssets > 0) {
            SafeTransferLib.safeTransferETH(_withdrawalVault,
    bufferedAssets);
```

```
            bufferedAssets = 0;
        }

        // Transfer staking rewards to the withdrawal vault
        if (address(_stakingRewardsVault).balance > 0) {
            uint256 rewardsToWithdraw
    = address(_stakingRewardsVault).balance;
            _collectRewards(rewardsToWithdraw);
            SafeTransferLib.safeTransferETH(_withdrawalVault,
    rewardsToWithdraw);
        }

        IWithdrawalVault(_withdrawalVault).notifyFullExitFromCL(_pubkey);
        if(msg.sender == _accountingOracle)
            IWithdrawalVault(_withdrawalVault).notifyFullExitFromCL(_pubkey);
        isFullyExited = true;
        _pause();

        emit FullExitTriggered();
    }
...
        (uint256 shares, bool isFullExitWithdraw, bool isShortCircuit) =
            IStakingPool

    (coreContracts.stakingPool).notifyWithdrawalRequest(msg.sender,
    amountInWei);

        // Only trigger withdrawals if pool hasn't fully exited and short
    circuit is not triggered
        if (!_isFullyExited[pubkey] && !isShortCircuit) {
            // For full exit withdrawals, set amount to 0 and mark as exited
            if (isFullExitWithdraw) {
                _isFullyExited[pubkey] = true;
                assetsInGWei = 0;
            }

            uint256 feeToPay = _getWithdrawalRequestFee();
            if (feeToPay > maxFeeToPay) {
                revert OverpaidFee();
            }

            if (msg.value < feeToPay) {
                revert InsufficientFee();
            }
```

```
            _triggerExecutionLayerWithdrawal(pubkey, assetsInGWei,
    feeToPay);
            _refundOverpaidFee(feeToPay, msg.sender);
        } else {
            // If the pool has fully exited or short circuit is triggered,
    refund the fee if any
            if (msg.value > 0) {
                SafeTransferLib.safeTransferETH(msg.sender, msg.value);
            }
        }
```

Berachain: Resolved with PR-59.

Zenith: Verified. Fixed as recommended.

[H-2] `AccountingOracle.updateTotalDeposits()` has no access control

SEVERITY: High	IMPACT: High
STATUS: Resolved	LIKELIHOOD: High

Target

- AccountingOracle.sol#L28-L46

Description:

`AccountingOracle.updateTotalDeposits()` has no access control, allowing anyone to update `stakingPool.totalDeposits` using the public proof.

```
function updateTotalDeposits(
    bytes memory pubkey,
    bytes32[] calldata pubkeyProof,
    uint64 balance,
    bytes32 balancesLeaf,
    bytes32[] calldata balanceProof,
    uint64 validatorIndex,
    uint64 timestamp
)
    external
{
    _factory.validatePubkeyProof(pubkey, pubkeyProof, validatorIndex,
    timestamp);
    _factory.validateBalanceProof(balance, balancesLeaf, balanceProof,
    validatorIndex, timestamp);

    uint256 balanceInWei = uint256(balance) * 1 gwei;
    ICoreContractsStorage.CoreContracts memory coreContracts
    = _factory.getCoreContracts(pubkey);
    address stakingPool = coreContracts.stakingPool;
    IStakingPool(stakingPool).updateTotalDeposits(balanceInWei);
}
```

The problem here is that when the stakingPool deposits to or withdraws from the validator, `stakingPool.totalDeposits` changes immediately, but the validator's balance does not

change immediately. This allows an attacker to use a stale proof (less than `MAX_TIMESTAMP_AGE`) to incorrectly update `stakingPool.totalDeposits`.

```
    function _depositToConsensusLayer(uint256 amount) internal {
        bytes memory _withdrawalCredentials = abi.encodePacked(bytes1(0x01),
    bytes11(0x0), _withdrawalVault);
        bytes memory _emptySignature = abi.encodePacked(bytes32(0),
    bytes32(0), bytes32(0));

        BEACON_DEPOSIT.deposit{ value: amount }(_pubkey,
    _withdrawalCredentials, _emptySignature, address(0));
        totalDeposits += amount;
    }
...
        if (!isFullyExited) {
            totalDeposits -= assets;
            if (totalDeposits < minEffectiveBalance()) {
                _triggerFullExit();
            }
```

Consider a user initiating a withdrawal, the protocol withdraws 10,000 Bera from the validator, reducing `totalDeposits` from 90,000 to 80,000.

The attacker then calls `AccountingOracle.updateTotalDeposits()` with a stale proof, updating `totalDeposits` to 90,000. `totalAssets` increases, and the attacker withdraws more assets.

Recommendations:

It is recommended to add access control to AccountingOracle.updateTotalDeposits() to prevent `totalDeposits` from being incorrectly updated by any user.

Berachain: Resolved with PR-62.**Zenith:** Verified. Fixed as recommended.

4.2 Medium Risk

A total of 4 medium risk findings were identified.

[M-1] Missing function in SmartOperator prevents IncentiveCollector fee percentage changes

SEVERITY: Medium	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- SmartOperator.sol
- IncentiveCollector.sol

Description:

The `IncentiveCollector` contract contains a `queueFeePercentageChange()` function that cannot be called due to a missing corresponding function in `SmartOperator`, making fee percentage updates impossible after deployment.

The `IncentiveCollector.queueFeePercentageChange()` function requires the caller to be the SmartOperator:

```
function queueFeePercentageChange(uint96 newFeePercentage) external {
    _validateSender(_smartOperator); // Only SmartOperator can call this
    emit QueuedFeePercentage(newFeePercentage, feePercentage);
    queuedFeePercentage = newFeePercentage;
}
```

However, the `SmartOperator` contract provides `queueIncentiveCollectorPayoutAmountChange()` for payout amount changes but has no equivalent function for fee percentage changes. This creates a broken flow where the queuing mechanism exists but cannot be triggered.

This results in the IncentiveCollector fee percentage being permanently fixed at its deployment value == 0, preventing any future adjustments to fee structures.

Recommendations:

Add the missing function to `SmartOperator.sol`:

```
function queueIncentiveCollectorFeePercentageChange(uint96 newFeePercentage)

    external
    onlyRole(INCENTIVE_COLLECTOR_MANAGER_ROLE)
{

    IIncentiveCollector(_incentiveCollector).queueFeePercentageChange(newFeePercentag
}
```

Berachain: Resolved with PR-64.

Zenith: Verified. Fixed by using `SmartOperator.protocolFeePercentage()` to set `IncentiveCollector.feePercentage()` in case the values did not match during the `IncentiveCollector.claim()` call.

[M-2] Double fee charging through share dilution after full exit causes user fund loss

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- SmartOperator.sol
- StakingPool.sol

Description:

The `SmartOperator` contract contains an issue that allows double fee charging through share dilution after a full exit is triggered, leading to unfair distribution of assets among remaining users.

When `_triggerFullExit()` is called, the `fullExitQueueDropBoost()` function resets `_bgtBalanceAlreadyCharged` to 0. However, during the time window between full exit and successful BGT redemption, the fee accrual functions remain callable:

1. Anyone can call `accrueEarnedBGTFees()` (no access control)
2. `PROTOCOL_FEE_MANAGER_ROLE` can call `setProtocolFeePercentage()` which triggers `_updateEarnedBGTFees()`

The problematic sequence occurs:

```
function fullExitQueueDropBoost() external {
    ...
    _bgtBalanceAlreadyCharged = 0;
}

function accrueEarnedBGTFees() external {
    _updateEarnedBGTFees();
    _bgtBalanceAlreadyCharged = BGT.balanceOf(address(this));
}
```

When `_updateEarnedBGTFees()` is subsequently called, the `_accountFees()` is triggered for whole BGT balance as `_bgtBalanceAlreadyCharged` was reset to 0:

```
function _updateEarnedBGTFees() internal {
    uint256 currentBalance = BGT.balanceOf(address(this));
    uint256 chargeableBalance = currentBalance - _bgtBalanceAlreadyCharged;

    if (protocolFeePercentage > 0) {
        uint256 feeAmount = _computeFee(chargeableBalance);
        _accountFees(feeAmount);
    }
}
```

This results in:

- Fees are charged twice on the same BGT balance since they were already properly charged before the reset
- Additional shares are minted to `_defaultSharesRecipient` through `_accountFees()`, diluting all existing shareholders
- Each user withdrawal after full exit reduces `_frozenTotalAssets`, making the share dilution more significant relative to remaining assets
- The later this attack occurs (but before BGT redemption), the greater the impact on remaining users

The attack window exists between full exit being triggered (when `_bgtBalanceAlreadyCharged` is reset to 0) and BGT being successfully redeemed via `fullExitRedeemBGT()` or `redeemBGT()`.

This creates an unfair situation where protocol fees are effectively charged twice on the same BGT balance, the `_defaultSharesRecipient` receives undeserved additional shares, all other shareholders bear the cost of this dilution, and users who withdraw later are disproportionately affected.

Recommendations:

Add the `whenNotFullyExited` modifier to the `accrueEarnedBGTFees()` and `setProtocolFeePercentage()` functions to prevent fee accrual after full exit. Also, resetting `_bgtBalanceAlreadyCharged` feels unneeded.

Berachain: Resolved with PR-63.

Zenith: Verified.

[M-3] BGT balance undervaluation during full exit leads to user fund loss

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- StakingPool.sol
- SmartOperator.sol

Description:

The `StakingPool` contains an issue in its full exit flow that leads to undervaluation of BGT assets, resulting in users receiving less value during withdrawals after a full exit is triggered.

When `notifyWithdrawalRequest()` is called and `totalDeposits < minEffectiveBalance()`, the system triggers `_triggerFullExit()`. This function performs the following sequence:

1. Calls `ISmartOperator(_smartOperator).fullExitQueueDropBoost()`
2. Immediately after, calls `_frozenTotalAssets = _getTotalAssets()`

The problematic part is in the `fullExitQueueDropBoost()` function in SmartOperator.sol. At the end of this function (line 135), `_bgtBalanceAlreadyCharged` is reset to 0:

```
function fullExitQueueDropBoost() external {
    ...
    _bgtBalanceAlreadyCharged = 0;
}
```

This reset is problematic because when `_getTotalAssets()` is subsequently called to freeze the total assets, it includes `ISmartOperator(_smartOperator).rebaseableBgtAmount()` in its calculation:

```
function rebaseableBgtAmount() external view returns (uint256) {
    uint256 balance = BGT.balanceOf(address(this));
    uint256 chargableBgtBalance = balance - _bgtBalanceAlreadyCharged;
    return balance - _computeFee(chargableBgtBalance);
}
```

Since `_bgtBalanceAlreadyCharged` was reset to 0, `chargableBgtBalance` equals the full BGT balance, causing the protocol fee to be deducted from the entire balance. However, protocol fees should have already been accounted for by the previous `_updateEarnedBGTFees()` call.

This results in:

- Undervalued `_frozenTotalAssets` stored as the "point of truth" for user withdrawals
- Users receiving less value than they should when withdrawing after a full exit
- Effective double-charging of protocol fees on the BGT balance

The `_frozenTotalAssets` serves as the basis for all subsequent user withdrawal calculations when `isFullyExited` is true, making this undervaluation persistent and affecting all users who withdraw after the full exit.

Recommendations:

Modify `fullExitQueueDropBoost()` to not reset `_bgtBalanceAlreadyCharged`, as the reset appears unnecessary during full exit since no further BGT operations will affect `_frozenTotalAssets`.

Berachain: Resolved with PR-63.

Zenith: Verified.

[M-4] Share inflation attack allows malicious staking pool creator to steal user deposits

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingPool.sol
- StBERA.sol

Description:

Anyone can create a StakingPool, making the StakingPool creator an untrusted actor. A malicious `_defaultSharesRecipient` (staking pool creator controlled address) can exploit the StakingPool/StBERA share calculation mechanism to steal funds from subsequent depositors by inflating the share price through strategic donations and share burning.

The attack leverages the fact that `_getTotalAssets()` includes `address(_stakingRewardsVault).balance` in its calculation, which can be inflated through direct donations to the StakingRewardsVault.

Attack scenario:

1. Pool activation: StakingPool is activated with `initialDepositAmount` (e.g., 10,000 ether)
   - `totalDeposits = 10,000 ether`
   - `_defaultSharesRecipient` receives 10,000 ether worth of shares
   - `totalSupply() = 10,000 ether`, `totalAssets() = 10,000 ether`
2. Donation and asset collection loop: Attacker repeatedly cycles:

   Step A - Donation: Send constrained amount to `StakingRewardsVault` (e.g., 200,000 ether)

   Step B - Minimal deposit with rewards collection: Call `submit(1 wei)`
   - `_collectRewards()` moves 200,000 ether from vault to `bufferedAssets`
   - Only 1 wei worth of deposits leads to 0 of shares minted (≈ 0 due to rounding)
   - `bufferedAssets` now contains 200,000 ether (but _canDeposit() returns false since 210,000 < 250,000 minEffectiveBalance)

   Step C - Share burning via withdrawal: Call `WithdrawalVault.requestRedeem()` which triggers `notifyWithdrawalRequest()`

- Since `!activeThresholdReached` and `bufferedAssets ⩾ assets`, the short-circuit path is taken
- Returns `isShortCircuit = true`, causing immediate withdrawal with no execution layer withdrawal triggered
- No pending withdrawal delay or `finalizeWithdrawalRequest()` needed - funds transferred instantly
- Burns attacker's shares proportional to withdrawn amount (e.g., withdraw 200,000 ether)
- The donated value effectively gets "absorbed" into the pool's asset base while burning shares

3. Gradual share inflation: Repeat loop to:
   - Move donated funds into the pool system via `bufferedAssets` collection
   - Mint minimal shares (1 wei deposits round to ~0 shares)
   - Burn initial shares through strategic withdrawals
   - Gradually reduce `totalSupply()` to near 1 wei while inflating `totalAssets()`
4. Victim exploitation: When legitimate users deposit:
   - Victims receive 0 shares but their funds increase `totalAssets()`
   - Attacker's remaining fractional shares gain claim to victim deposits

Once the share price is inflated, the attacker can exploit unlimited subsequent depositors, effectively stealing all future deposits with zero-share minting.

Recommendations:

Consider adding validation in `_burnShares()` to prevent total share supply from falling below the initial activation amount (10,000e18) unless the staking pool is in full exit mode.

Berachain: Resolved with PR-60.

Zenith: Verified.

4.3 Low Risk

A total of 5 low risk findings were identified.

[L-1] Unfair reward distribution in IncentiveCollector due to incorrect fee accounting order

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- IncentiveCollector.sol
- SmartOperator.sol

Description:

The `IncentiveCollector.claim()` function contains a timing issue that results in unfair distribution of incentive rewards, giving the fee recipient more value than intended at the expense of existing shareholders.

When `claim()` is called, the following sequence occurs:

```
function claim(address[] calldata tokens) external payable {
    ...
    uint256 fee = _computeFee(payoutAmount);
    ISmartOperator(_smartOperator).accrueIncentivesFees(fee); // ⟵ Mints
    fee shares first

    SafeTransferLib.safeTransferETH(_stakingRewardsVault, payoutAmount); //
    ⟵ Transfers full amount after
    ...
}
```

The problematic order of operations:

1. Fee shares are minted first when `accrueIncentivesFees(fee)` calls `_accountFees()` which mints shares equivalent to the fee amount to `_defaultSharesRecipient`
2. The entire `payoutAmount` (including the fee portion) is transferred to the staking rewards

vault after

Since `_getTotalAssets()` includes `address(_stakingRewardsVault).balance` in its calculation, when the full payout is transferred to the rewards vault, it increases the value per share for all shareholders, including the newly minted fee shares.

This results in the fee recipient receiving:

- Fee shares based on the fee amount (intended compensation)
- Additional value from their proportional share of the entire payout transfer (unintended extra benefit)

Meanwhile, existing shareholders receive less value than they should because:

- They should benefit from the full (payoutAmount - fee)
- But the fee recipient also gets a proportional share of this amount through their newly minted shares

The impact is that incentive rewards that should belong solely to current shareholders at the time of claim are partially redistributed to the fee recipient, creating an unfair advantage for the protocol fee collector.

Recommendations:

Modify the order of operations to ensure fair distribution:

```
function claim(address[] calldata tokens) external payable {
    ...
    uint256 fee = _computeFee(payoutAmount);
    uint256 netPayout = payoutAmount - fee;

    // Transfer net amount to existing shareholders first
    SafeTransferLib.safeTransferETH(_stakingRewardsVault, netPayout);

    // Then mint fee shares based on fee amount
    ISmartOperator(_smartOperator).accrueIncentivesFees(fee);

    // Transfer fee amount separately
    SafeTransferLib.safeTransferETH(_stakingRewardsVault, fee);
    ...
}
```

Berachain: Resolved with PR 57.

Zenith: Verified.

[L-2] Dust assets cannot be withdrawn

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- WithdrawalVault.sol#L95-L126

Description:

The amount of assets to be withdrawn must be a multiple of 1 GWei, which means that dust assets cannot be withdrawn.

```
function requestRedeem(
    bytes memory pubkey,
    uint256 shares,
    uint256 maxFeeToPay
)
    external
    payable
    nonReentrant
    whenNotPaused
{
    ICoreContractsStorage.CoreContracts memory coreContracts
    = _factory.getCoreContracts(pubkey);
    uint256 assetsInWei
    = IStBera(coreContracts.stakingPool).previewRedeem(shares);

    // Round down to the nearest GWei
    uint64 assetsInGWei = uint64(assetsInWei / 1 gwei);

    _withdraw(pubkey, assetsInGWei, maxFeeToPay);
}

/// @inheritdoc IWithdrawalVault
function requestWithdrawal(
    bytes memory pubkey,
    uint64 assetsInGWei,
    uint256 maxFeeToPay
)
```

```
    external
    payable
    nonReentrant
    whenNotPaused
{
    _withdraw(pubkey, assetsInGWei, maxFeeToPay);
}
```

Recommendations:

One option is to withdraw dust assets from `bufferedAssets`, or just document the issue to inform users.

Berachain: Acknowledged. Added docs in PR-61/files#diff-ef0835cf21d29e1fc318c117a56b6ac1877f1e183401b1c9e36884668b0e9a99R84)

[L-3] Withdrawals may fail due to exceeding `totalDeposits`

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingPool.sol#L168-L173

Description:

When a user withdraws funds, if `activeThresholdReached` is true or `assets > bufferedAssets` and the validator has not fully exited, the protocol will only withdraw from the `totalDeposits` without considering `bufferedAssets`.

```
if (!isFullyExited) {
    totalDeposits -= assets;
    if (totalDeposits < minEffectiveBalance()) {
        _triggerFullExit();
    }
}
```

1. Consider the deployer first depositing 10,000 Bera to activate the validator, `totalDeposits` == 10,000.

2. Subsequently, the deployer deposits 5000 Bera. Since this does not meet the minimum deposit amount, `bufferedAssets` is set to 5000.

3. Later, the deployer attempts to withdraw 15000 Bera. The protocol will directly withdraw from `totalDeposits`, causing `totalDeposits -= assets` to underflow, resulting in the withdrawal to fail.

Recommendations:

One option is to modify the withdrawal logic, considering withdrawing from bufferedAssets. Or just document the issue to inform users.

Berachain: Resolved with PR-60.

Zenith: Verified. The fix allows withdrawing assets from `bufferedAssets` when the full exit is triggered.

[L-4] `StBERA.previewRedeem()` should round down

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- StBERA.sol#L87-L89

Description:

`StBERA.previewRedeem()` is used to calculate the shares to assets amount. But according to the specification, it should round down.

```
function previewRedeem(uint256 shares) public view virtual returns (uint256)
    {
    return _convertToAssets(shares, Math.Rounding.Ceil);
}
```

Otherwise, in `requestRedeem()`, the calculated shares may exceed the user provided shares, resulting in the withdrawal to fail.

```
function requestRedeem(
    bytes memory pubkey,
    uint256 shares,
    uint256 maxFeeToPay
)
    external
    payable
    nonReentrant
    whenNotPaused
{
    ICoreContractsStorage.CoreContracts memory coreContracts
    = _factory.getCoreContracts(pubkey);
    uint256 assetsInWei
    = IStBera(coreContracts.stakingPool).previewRedeem(shares);

    // Round down to the nearest GWei
    uint64 assetsInGWei = uint64(assetsInWei / 1 gwei);
```

```
    _withdraw(pubkey, assetsInGWei, maxFeeToPay);
}
```

Recommendations:

```
function previewRedeem(uint256 shares) public view virtual returns (uint256)
    {
    return _convertToAssets(shares, Math.Rounding.Ceil);
    return _convertToAssets(shares, Math.Rounding.Floor);
}
```

Berachain: Resolved with PR-56.

Zenith: Verified.

[L-5] Unnecessary `_safeMint()` can block withdrawals for contracts not supporting `onERC721Received()`

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- WithdrawalVault.sol

Description:

The `_mintWithdrawalRequest()` function uses `_safeMint()` to create withdrawal request NFTs, which can prevent contracts from withdrawing their deposited funds if they don't implement the ERC721 receiver interface.

`_safeMint()` calls `onERC721Received()` on the recipient contract. If the contract doesn't implement this interface, the call reverts, preventing withdrawal request creation and effectively locking the contract's funds.

The ERC721 receiver check is unnecessary because:

1. The NFT is non-transferable (`transferFrom()` always reverts)
2. Anyone can call `finalizeWithdrawalRequest()` for any request ID
3. The withdrawal system works without any NFT interaction

This creates a situation where contracts that deposited funds could be permanently unable to withdraw them.

Recommendations:

Replace `_safeMint()` with `_mint()`.

Berachain: Resolved with PR-58.

Zenith: Verified.

4.4 Informational

A total of 12 informational findings were identified.

[I-1] EOA users unprotected against race conditions in IncentiveCollector claim function

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- IncentiveCollector.sol

Description:

The `IncentiveCollector.claim()` function lacks built-in slippage protection, creating potential fund loss for EOA users despite developer documentation acknowledging this limitation.

The interface contains a developer comment stating:

```
/// @dev This function is NOT implementing slippage protection. Caller has
    to check that received amounts match the minimum expected.
```

However, the suggested mitigation of "caller has to check" is not feasible for standard EOA users who cannot perform post-transaction validation and reverting within the same transaction.

The race condition scenario remains:

1. Multiple users call `claim()` simultaneously with `payoutAmount` ETH
2. First caller receives all available tokens
3. Subsequent callers receive zero tokens but still pay full amount
4. EOA users cannot revert the transaction based on received token amounts

Recommendations:

Consider adding optional slippage protection parameters to make the function safer for EOA users:

```
function claimWithSlippage(address[] calldata tokens,
    uint256[] calldata minAmounts) external payable {
    // Implementation with minimum amount checks
}
```

Berachain: Acknowledged.

[I-2] Add validation to prevent zero beacon block root usage in proof verification

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- BeaconRootsHelper.sol

Description:

The `_getBeaconBlockRoot()` function returns beacon block roots from EIP-4788 without validating against `bytes32(0)`, potentially allowing proof validation bypass in edge cases.

According to EIP-4788, the beacon block root precompile doesn't enforce that calldata for a given timestamp must be non-zero. If the system address fails or returns empty data, `timestamp.getParentBlockRootAt()` could return `bytes32(0)`.

The returned beacon block root is used in critical proof verification functions:

- `_verifyValidatorPubkeyInBeaconBlock()`
- `_verifyValidatorWithdrawalCredentialsInBeaconBlock()`
- `_verifyValidatorBalanceInBeaconBlock()`

If a zero beacon block root is passed to these verification functions, it could potentially bypass proof validation checks.

While this scenario would require system-level failures or edge cases in the beacon block root precompile, the validation functions should defensively reject zero roots to prevent any possibility of proof bypass.

Recommendations:

Add a validation check in `_getBeaconBlockRoot()` to reject zero beacon block roots:

```
function _getBeaconBlockRoot(uint64 timestamp)
    internal view returns (bytes32) {
    bytes32 root = timestamp.getParentBlockRootAt();
    if (root == bytes32(0)) {
        revert InvalidBeaconBlockRoot();
```

```
    }
    return root;
}
```

Berachain: Resolved with PR-65.

Zenith: Verified. Fixed as recommended.

[I-3] Incorrect comment and redundant code in `accrueEarnedBGTFees()` function

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- SmartOperator.sol

Description:

The `accrueEarnedBGTFees()` function contains an incorrect comment and redundant code that was incorrectly copied from the `redeemBGT()` function.

The comment describes BGT redemption behavior: "When BGTs are redeemed, they are burned and the equivalent BERA is deposited into the staking pool." However, `accrueEarnedBGTFees()` doesn't perform any redemption operations - it only calls `_updateEarnedBGTFees()` to calculate and mint fee shares.

The line `_bgtBalanceAlreadyCharged = BGT.balanceOf(address(this))` is also redundant in this context since there's no redemption happening that would require updating the already-charged balance tracking.

Recommendations:

Remove the incorrect comment and redundant `_bgtBalanceAlreadyCharged` assignment from `accrueEarnedBGTFees()`. The function should only call `_updateEarnedBGTFees()`.

Berachain: Resolved with PR-63.

Zenith: Verified.

[I-4] Update NatSpec documentation and inheritance docs for consistency

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- IStakingPool.sol
- ISmartOperator.sol
- IIncentiveCollector.sol
- StakingPool.sol
- SmartOperator.sol
- IncentiveCollector.sol

Description:

During development, many inconsistencies were introduced to NatSpec documentation and inheritance docs. The documentation doesn't correctly reflect the final code implementation.

Missing @inheritdoc tags:

- SmartOperator functions missing `@inheritdoc` (queueDropBoost, etc.)
- IncentiveCollector functions missing `@inheritdoc` (queueFeePercentageChange)

Outdated references:

- IStakingPool references "withdrawalRequestERC721" instead of "WithdrawalVault"
- ISmartOperator mentions "WithdrawalRequestERC721 contract" but logic moved to WithdrawalVault
- IIncentiveCollector says "admin" instead of "smart operator contract"

Incorrect error references:

- Multiple locations reference "TransferFailed" but should be "ETHTransferFailed"

Missing function declarations:

- Some SmartOperator functions not declared in interface

Incomplete NatSpec:

- Missing parameter documentation in ISmartOperator
- Missing return value documentation
- Incorrect behavior descriptions (contract pausing, fee deduction timing)

Recommendations:

Update all NatSpec documentation to reflect current implementation, add missing `@inheritdoc` tags, correct outdated references, and ensure interface declarations match implementations.

Berachain: Resolved with PR-61.

Zenith: Verified.

[I-5] Remove redundant declarations and duplicate imports

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- IWithdrawalVault.sol
- IStakingPoolContractsFactory.sol
- ISmartOperator.sol
- IIncentiveCollector.sol
- StakingPool.sol

Description:

The codebase contains several redundant declarations, unused events/errors, and duplicate imports that should be cleaned up for better maintainability.

Unused events and errors:

- `WithdrawalsWithdrawn` event and `WithdrawalFailed` error in IWithdrawalVault.sol
- `WithdrawalVaultBeaconImplUpgraded` event in IStakingPoolContractsFactory.sol
- `ProtocolFeeCollected` event, `InvalidAmount` and `TransferFailed` errors in ISmartOperator.sol
- `TransferFailed` error in IIncentiveCollector.sol

Duplicate imports in StakingPool.sol:

- `IStakingRewardsVault` imported twice
- `ISmartOperator` imported twice

Re-declaration:

- `getCoreContracts()` function redeclared in IStakingPoolContractsFactory.sol when already available in ICoreContractsStorage

Recommendations:

Remove unused events, errors, and duplicate imports. Consolidate interface declarations to avoid redundancy.

Berachain: Resolved with PR-61.

Zenith: Verified.

[I-6] Restrict `recoverERC20()` to only work after full exit in `IncentiveCollector` contract

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- IncentiveCollector.sol

Description:

The `recoverERC20()` function can be called by DEFAULT_ADMIN_ROLE at any time, even while the staking pool is active. During active operation, all ERC20 tokens in the IncentiveCollector belong to users and should only be claimable via the `claim()` function for the payoutAmount fee.

The `claim()` function correctly prevents access after full exit, but `recoverERC20()` has no such restriction. This allows admin to potentially withdraw incentive tokens before they can be claimed. Additionally, the recovery function is largely ineffective since any valuable tokens would already be claimed by users via the `claim()` mechanism.

Recommendations:

Add a check to restrict `recoverERC20()` to only work after full exit.

Additionally, use `SafeERC20.safeTransfer()` instead of `transfer()` to handle non-standard ERC20 in `recoverERC20()` function.

Berachain: Acknowledged. In PR-68 the only change introduced was the use of `SafeERC20.safeTransfer()`.

[I-7] Missing explicit `requestId` validation in `_finalizeWithdrawalRequest()`

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- WithdrawalVault.sol

Description:

The `_finalizeWithdrawalRequest()` function lacks explicit validation for `requestId` validity. Invalid requestIds are processed as phantom requests with default values until `_burn(requestId)` fails when trying to burn a non-existent NFT.

This provides unclear error messaging, rather than a descriptive error about the invalid requestId.

Recommendations:

Add explicit requestId validation at the beginning of `_finalizeWithdrawalRequest()`.

Berachain: Fixed in PR-66.

Zenith: Verified. Fixed as recommended.

[I-8] Change public functions to external

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingPoolContractsFactory.sol

Description:

Several functions are declared as `public` but are only called externally.

StakingPoolContractsFactory:

- `setZeroValidatorPubkeyGIndex()`
- `setZeroValidatorWithdrawalCredentialsGIndex()`
- `setZeroValidatorBalanceGIndex()`
- `validatePubkeyProof()`
- `validateBalanceProof()`

Recommendations:

Change function visibility from `public` to `external`.

Berachain: Resolved with PR-67.

Zenith: Verified.

[I-9] Use `Ownable2StepUpgradeable` instead of `OwnableUpgradeable`

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- StakingPoolContractsFactory.sol
- StakingPool.sol
- AccountingOracle.sol

Description:

The protocol uses `OwnableUpgradeable` for access control in critical contracts that handle upgrades, beacon implementations, and emergency functions.

`OwnableUpgradeable` uses single-step ownership transfer via `transferOwnership()`, which can lead to accidental ownership loss if transferred to wrong address (typos, wrong network). There's no recovery mechanism once transfer is completed.

Recommendations:

Replace `OwnableUpgradeable` with `Ownable2StepUpgradeable`.

Berachain: Resolved with PR-71.

Zenith: Verified.

[I-10] Add clarification that direct `BEACON_DEPOSIT` donations are considered a loss

`SEVERITY:` Informational	`IMPACT:` Informational
`STATUS:` Acknowledged	`LIKELIHOOD:` Low

Target

- StakingPoolContractsFactory.sol
- StakingPool.sol
- AccountingOracle.sol

Description:

The staking pool activation mechanism contains a race condition problem that is only exploitable when direct extra deposits have been made to `BEACON_DEPOSIT` between contract deployment and activation.

In rare situations, a staking pool creator might deposit directly to `BEACON_DEPOSIT` to mint more initial shares with a higher balance proof. However, this can be frontrun using older proofs that are still valid within the 10 minute timestamp validation window for using beacon block roots, and depositing to staking pool right after the activation.

Later, when `AccountingOracle.updateTotalDeposits()` updates to the real balance, the extra BERA becomes "gains" distributed to existing shareholders instead of minting additional initial shares.

Recommendations:

Add documentation clarifying that direct deposits to a pubkey via `BEACON_DEPOSIT` are considered a loss for the depositor and a direct gain for shareholders.

Berachain: Acknowledged. We will add this warning to the official documentation and guides on how to use this protocol.

[I-11] Incorrect deposit prioritization leads to rewards dilution for existing users

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- StakingPool.sol

Description:

The `_computeDepositAmounts()` function incorrectly prioritizes new user deposits over rewards collection when the staking pool is near maximum capacity. This leads to rewards dilution for existing users.

When the pool has limited remaining capacity (`additionalDeposit < userDepositAmount`), the current logic gives the entire additional capacity to the new user deposit while leaving rewards uncollected in the rewards vault. This is problematic because rewards represent compounding returns that belong to existing users and should be prioritized over new deposits.

This means when capacity is limited, new deposits are fully accommodated while rewards that should compound for existing users remain in the rewards vault, unable to be collected and compounded into the pool's total assets.

Recommendations:

Modify the deposit calculation logic to prioritize rewards collection over new user deposits:

1. First allocate available capacity to rewards collection (up to `rewardsBalance`)
2. Then allocate remaining capacity to user deposits
3. Only refund user deposits for the portion that cannot fit after rewards are prioritized

Solv: Acknowledged.

Zenith: Partially addressed with a new function to compound rewards.

[I-12] The Deployer contract may always fail to deploy

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- Deployer.sol#L22-L46

Description:

In the Deployer contract's constructor, it always uses a fixed salt of 0 to deploy the AccountingOracle/StakingPoolContractsFactory/WithdrawalVault implementation.

```
constructor(
    address governance,
    uint256 accountingOracleSalt,
    uint256 withdrawalVaultSalt,
    uint256 stakingPoolContractsFactorySalt,
    address smartOperatorInitialImpl,
    address stakingPoolInitialImpl,
    address stakingRewardsVaultInitialImpl,
    address incentiveCollectorInitialImpl
) {
    // Deploy accounting oracle implementation
    address accountingOracleImpl = deployWithCreate2(0,
    type(AccountingOracle).creationCode);
    // deploy accounting oracle proxy
    accountingOracle = AccountingOracle
        (deployProxyWithCreate2(accountingOracleImpl,
    accountingOracleSalt));

    // Deploy staking pool contracts factory implementation
    address stakingPoolContractsFactoryImpl = deployWithCreate2
            (0, type(StakingPoolContractsFactory).creationCode);

    // deploy staking pool contracts factory proxy
    stakingPoolContractsFactory = StakingPoolContractsFactory(
        deployProxyWithCreate2
            (stakingPoolContractsFactoryImpl,
    stakingPoolContractsFactorySalt)
```

```
    );

    // Deploy withdrawal vault implementation
    address withdrawalVaultImpl = deployWithCreate2(0,
    type(WithdrawalVault).creationCode);
```

`deployWithCreate2()` calls `_CREATE2_FACTORY` to deploy the contract, which uses the create2 opcode to create the contract. This means that the caller address when creating the contract is `_CREATE2_FACTORY`, and anyone can deploy the contract by calling `_CREATE2_FACTORY` with the same arguments. If an attacker creates these contracts with the salt of 0, the Deployer constructor will always fail.

```
function deployWithCreate2(uint256 salt, bytes memory initCode) internal
    returns (address addr) {
    assembly ("memory-safe") {
        // cache the length of the init code
        let length := mload(initCode)
        // overwrite the length memory slot with the salt
        mstore(initCode, salt)
        // deploy the contract using the _CREATE2_FACTORY
        if iszero(call(gas(), _CREATE2_FACTORY, 0, initCode, add(length,
    0x20), 0, 0x14)) {
            mstore(0, 0x30116425) // selector for DeploymentFailed()
            revert(0x1c, 0x04)
        }
        addr := shr(96, mload(0))
        // restore the length memory slot
        mstore(initCode, length)
    }
}
```

PoC:

```
function test_Deploy() public {
    vm.prank(makeAddr("123"));
    address SOImpl =
        payable(deployWithCreate2(1, type(SmartOperator).creationCode));
    console2.log(SOImpl);
    vm.prank(makeAddr("456"));
    SOImpl = payable
        (deployWithCreate2(1, type(SmartOperator).creationCode)); //
    DeploymentFailed
    console2.log(SOImpl);
}
```

Recommendations:

It is recommended to reuse the proxy's salt value to deploy the implementation.

```
constructor(
    address governance,
    uint256 accountingOracleSalt,
    uint256 withdrawalVaultSalt,
    uint256 stakingPoolContractsFactorySalt,
    address smartOperatorInitialImpl,
    address stakingPoolInitialImpl,
    address stakingRewardsVaultInitialImpl,
    address incentiveCollectorInitialImpl
) {
    // Deploy accounting oracle implementation
    address accountingOracleImpl = deployWithCreate2
        (0, type(AccountingOracle).creationCode);
    address accountingOracleImpl = deployWithCreate2
        (accountingOracleSalt, type(AccountingOracle).creationCode);
    // deploy accounting oracle proxy
    accountingOracle
    = AccountingOracle(deployProxyWithCreate2(accountingOracleImpl,
    accountingOracleSalt));

    // Deploy staking pool contracts factory implementation
    address stakingPoolContractsFactoryImpl = deployWithCreate2
            (0, type(StakingPoolContractsFactory).creationCode);
    address stakingPoolContractsFactoryImpl = deployWithCreate2
        (stakingPoolContractsFactorySalt, type(StakingPoolContractsFactory).
            creationCode);

    // deploy staking pool contracts factory proxy
    stakingPoolContractsFactory = StakingPoolContractsFactory(
        deployProxyWithCreate2
            (stakingPoolContractsFactoryImpl,
    stakingPoolContractsFactorySalt)
    );

    // Deploy withdrawal vault implementation
    address withdrawalVaultImpl =
    deployWithCreate2(0, type(WithdrawalVault).creationCode);
    address withdrawalVaultImpl = deployWithCreate2
        (withdrawalVaultSalt, type(WithdrawalVault).creationCode);
```

Berachain: Resolved with PR-69.

Zenith: Verified. Fixed as recommended.